

RNN Examples: Detailed Step-by-Step Calculations

Character-Level Language Model, Image Captioning, and Gradient Flow

Table of Contents

1. [Character-Level Language Model - Complete Example](#)
 2. [Image Captioning - Complete Example](#)
 3. [Vanilla RNN Gradient Flow - Complete Example](#)
-

1. Character-Level Language Model

1.1 Problem Statement

Task: Train an RNN to predict the next character in a sequence.

Example Training Sequence: "hello"

Vocabulary: [h, e, l, o] (4 characters)

Goal: Learn $P(\text{next_char} \mid \text{previous_chars})$

1.2 Architecture Setup

Dimensions:

- Vocabulary size (input/output): 4
- Hidden state size: 3
- Embedding dimension: 2

Network Structure:

Input (one-hot) → Embedding Layer → RNN → Softmax → Output (probability distribution)

1.3 Step-by-Step Training Example

Step 1: Data Preparation

Training sequence: "hello"

Character to Index Mapping:

```
h → 0
e → 1
l → 2
o → 3
```

Create Input-Output Pairs:

```
Time 0: Input = <START>, Target = 'h' (index 0)
Time 1: Input = 'h' (index 0), Target = 'e' (index 1)
Time 2: Input = 'e' (index 1), Target = 'l' (index 2)
Time 3: Input = 'l' (index 2), Target = 'l' (index 2)
Time 4: Input = 'l' (index 2), Target = 'o' (index 3)
```

Step 2: Initialize Weights

Embedding Matrix W_{embed} (4×2):

```
W_embed = [[0.2, -0.1], # for 'h'
            [0.3, 0.2],  # for 'e'
            [-0.1, 0.4], # for 'l'
            [0.1, -0.2]] # for 'o'
```

RNN Weights:

```
W_xh (2×3) = [[0.5, -0.3, 0.2],
               [0.1, 0.4, -0.1]]
```

```
W_hh (3×3) = [[0.3, -0.2, 0.1],
               [0.2, 0.4, -0.3],
               [-0.1, 0.2, 0.3]]
```

```
b_h (3×1) = [0.1, 0.0, -0.1]
```

Output Weights:

```
W_hy (3×4) = [[0.4, -0.2, 0.3, 0.1],
               [0.2, 0.5, -0.1, 0.3],
               [-0.3, 0.1, 0.4, -0.2]]
```

```
b_y (4×1) = [0.0, 0.1, -0.1, 0.0]
```

Step 3: Forward Pass - Time Step 1

Input: 'h' (index 0)

Step 3.1: Embedding

One-hot encoding of 'h':

```
x_1 = [1, 0, 0, 0]
```

Embedding lookup (simply extract row 0 of W_embed):

```
e_1 = W_embed[0] = [0.2, -0.1]
```

Step 3.2: Hidden State Calculation

Initialize: $h_0 = [0, 0, 0]$

```
h_1 = tanh(W_xh^T · e_1 + W_hh^T · h_0 + b_h)
```

```
W_xh^T · e_1 = [[0.5, 0.1],      [[0.2],      [[0.09],
               [-0.3, 0.4],    ×  [-0.1]] =  [-0.10],
               [0.2, -0.1]]      [0.05]]
```

```
W_hh^T · h_0 = [[0, 0, 0], (all zeros since h_0 = 0)
```

```
h_1 = tanh([0.09, -0.10, 0.05] + [0, 0, 0] + [0.1, 0.0, -0.1])
```

```
h_1 = tanh([0.19, -0.10, -0.05])
```

```
h_1 = [0.188, -0.100, -0.050]
```

Step 3.3: Output Calculation

$$y_1 = W_hy^T \cdot h_1 + b_y$$

$$W_hy^T = \begin{bmatrix} 0.4 & 0.2 & -0.3 \\ -0.2 & 0.5 & 0.1 \\ 0.3 & -0.1 & 0.4 \\ 0.1 & 0.3 & -0.2 \end{bmatrix}$$

$$W_hy^T \cdot h_1 = \begin{bmatrix} 0.4 & 0.2 & -0.3 \\ -0.2 & 0.5 & 0.1 \\ 0.3 & -0.1 & 0.4 \\ 0.1 & 0.3 & -0.2 \end{bmatrix} \times \begin{bmatrix} 0.188 \\ -0.100 \\ -0.050 \end{bmatrix}$$

$$\text{Row 1: } 0.4(0.188) + 0.2(-0.100) + (-0.3)(-0.050) = 0.0752 - 0.020 + 0.015 = 0.0702$$

$$\text{Row 2: } -0.2(0.188) + 0.5(-0.100) + 0.1(-0.050) = -0.0376 - 0.050 - 0.005 = -0.0926$$

$$\text{Row 3: } 0.3(0.188) + (-0.1)(-0.100) + 0.4(-0.050) = 0.0564 + 0.010 - 0.020 = 0.0464$$

$$\text{Row 4: } 0.1(0.188) + 0.3(-0.100) + (-0.2)(-0.050) = 0.0188 - 0.030 + 0.010 = -0.0012$$

$$y_1 = [0.0702, -0.0926, 0.0464, -0.0012] + [0.0, 0.1, -0.1, 0.0]$$

$$y_1 = [0.0702, 0.0074, -0.0536, -0.0012]$$

Step 3.4: Softmax

$$\text{softmax}(y_1) = \exp(y_1) / \sum \exp(y_1)$$

$$\begin{aligned} \exp(y_1) &= [\exp(0.0702), \exp(0.0074), \exp(-0.0536), \exp(-0.0012)] \\ &= [1.0727, 1.0074, 0.9478, 0.9988] \end{aligned}$$

$$\sum \exp(y_1) = 1.0727 + 1.0074 + 0.9478 + 0.9988 = 4.0267$$

$$P_1 = [1.0727/4.0267, 1.0074/4.0267, 0.9478/4.0267, 0.9988/4.0267]$$

$$P_1 = [0.266, 0.250, 0.235, 0.248]$$

Interpretation: Probability distribution over next character:

$$P(\text{next} = 'h') = 0.266$$

$$P(\text{next} = 'e') = 0.250$$

$$P(\text{next} = 'l') = 0.235$$

$$P(\text{next} = 'o') = 0.248$$

Target: 'e' (index 1)

Step 3.5: Loss Calculation (Cross-Entropy)

```
L_1 = -log(P_1[1])  
L_1 = -log(0.250)  
L_1 = 1.386
```

Step 4: Forward Pass - Time Step 2

Input: 'e' (index 1) **Target:** 'l' (index 2)

Step 4.1: Embedding

```
e_2 = W_embed[1] = [0.3, 0.2]
```

Step 4.2: Hidden State

```
h_2 = tanh(W_xh^T · e_2 + W_hh^T · h_1 + b_h)
```

```
W_xh^T · e_2 = [[0.5, 0.1],      [[0.3],      [[0.17],  
                [-0.3, 0.4],    ×  [0.2]]    =  [0.01],  
                [0.2, -0.1]]    [0.04]]
```

```
W_hh^T · h_1 = [[0.3, 0.2, -0.1],  [[0.188],  
                [-0.2, 0.4, 0.2],    ×  [-0.100],  
                [0.1, -0.3, 0.3]]    [-0.050]]
```

Row 1: $0.3(0.188) + 0.2(-0.100) + (-0.1)(-0.050) = 0.0564 - 0.020 + 0.005 = 0.0414$

Row 2: $-0.2(0.188) + 0.4(-0.100) + 0.2(-0.050) = -0.0376 - 0.040 - 0.010 = -0.0876$

Row 3: $0.1(0.188) + (-0.3)(-0.100) + 0.3(-0.050) = 0.0188 + 0.030 - 0.015 = 0.0338$

```
h_2 = tanh([0.17, 0.01, 0.04] + [0.0414, -0.0876, 0.0338] + [0.1, 0.0, -0.1])
```

```
h_2 = tanh([0.3114, -0.0776, -0.0262])
```

```
h_2 = [0.302, -0.077, -0.026]
```

Step 4.3: Output and Softmax

$$y_2 = W_hy^{AT} \cdot h_2 + b_y$$

$$W_hy^{AT} \cdot h_2 = \begin{bmatrix} 0.4 & 0.2 & -0.3 \\ -0.2 & 0.5 & 0.1 \\ 0.3 & -0.1 & 0.4 \\ 0.1 & 0.3 & -0.2 \end{bmatrix} \times \begin{bmatrix} 0.302 \\ -0.077 \\ -0.026 \end{bmatrix}$$

$$\text{Row 1: } 0.4(0.302) + 0.2(-0.077) + (-0.3)(-0.026) = 0.1208 - 0.0154 + 0.0078 = 0.1132$$

$$\text{Row 2: } -0.2(0.302) + 0.5(-0.077) + 0.1(-0.026) = -0.0604 - 0.0385 - 0.0026 = -0.1015$$

$$\text{Row 3: } 0.3(0.302) + (-0.1)(-0.077) + 0.4(-0.026) = 0.0906 + 0.0077 - 0.0104 = 0.0879$$

$$\text{Row 4: } 0.1(0.302) + 0.3(-0.077) + (-0.2)(-0.026) = 0.0302 - 0.0231 + 0.0052 = 0.0123$$

$$y_2 = [0.1132, -0.1015, 0.0879, 0.0123] + [0.0, 0.1, -0.1, 0.0]$$

$$y_2 = [0.1132, -0.0015, -0.0121, 0.0123]$$

$$\text{softmax}(y_2):$$

$$\exp(y_2) = [1.1198, 0.9985, 0.9880, 1.0124]$$

$$\Sigma = 4.1187$$

$$P_2 = [0.272, 0.242, 0.240, 0.246]$$

Target: 'l' (index 2)

$$L_2 = -\log(P_2[2]) = -\log(0.240) = 1.427$$

Step 5: Sampling at Test Time

At test time, we sample characters one at a time and feed back:

Start: Input = 'h'

1. Get probability distribution $P_1 = [0.266, 0.250, 0.235, 0.248]$
2. Sample from distribution (e.g., sample 'e' with probability 0.250)
3. Feed 'e' as next input
4. Get probability distribution P_2
5. Sample next character
6. Repeat until end token or max length

Example Sampling:

```
Start with 'h'  
→ Sample 'e' (prob 0.250)  
→ Sample 'l' (prob 0.240)  
→ Sample 'l' (prob 0.235)  
→ Sample 'o' (prob 0.248)  
  
Generated sequence: "hello"
```

1.4 Complete Example Question with Answer

Question 1: Character-Level Language Model Training

Given:

- Vocabulary: ['a', 'b', 'c']
- Training sequence: "abc"
- Hidden dimension: 2
- Embedding dimension: 2

Weights:

```
W_embed = [[0.5, 0.2], # 'a'  
            [0.3, -0.1], # 'b'  
            [-0.2, 0.4]] # 'c'  
  
W_xh = [[0.6, -0.2],  
        [0.1, 0.5]]  
  
W_hh = [[0.4, 0.3],  
        [-0.2, 0.5]]  
  
W_hy = [[0.7, 0.2, -0.3],  
        [0.1, -0.4, 0.6]]  
  
b_h = [0.0, 0.0]  
b_y = [0.0, 0.0, 0.0]
```

Tasks: a) Calculate hidden state h_1 when input is 'a' (starting from $h_0 = [0, 0]$) b) Calculate output probabilities $P(\text{next_char})$ after seeing 'a' c) If target is 'b', calculate the loss d) Calculate hidden state h_2 when input sequence is "ab"

Solution:

a) Hidden state h_1 :

Input 'a' $\rightarrow$ index 0

$$e_1 = W_{\text{embed}}[0] = [0.5, 0.2]$$

$$h_1 = \tanh(W_{xh}^T \cdot e_1 + W_{hh}^T \cdot h_0 + b_h)$$

$$W_{xh}^T \cdot e_1 = \begin{bmatrix} [0.6, 0.1], & [[0.5], & [[0.32], \\ [-0.2, 0.5]] & \times [0.2]] & = [0.00] \end{bmatrix}$$

$$W_{hh}^T \cdot h_0 = [0, 0] \text{ (since } h_0 = 0)$$

$$h_1 = \tanh([0.32, 0.00] + [0.0, 0.0])$$

$$h_1 = \tanh([0.32, 0.00])$$

$$h_1 = [0.310, 0.000]$$

b) Output probabilities:

$$y_1 = W_{hy}^T \cdot h_1 + b_y$$

$$W_{hy}^T = \begin{bmatrix} [0.7, 0.1], \\ [0.2, -0.4], \\ [-0.3, 0.6] \end{bmatrix}$$

$$W_{hy}^T \cdot h_1 = \begin{bmatrix} [0.7, 0.1], & [[0.310], & [[0.217], \\ [0.2, -0.4], & \times [0.000]] & = [0.062], \\ [-0.3, 0.6] & & [-0.093] \end{bmatrix}$$

$$y_1 = [0.217, 0.062, -0.093] + [0, 0, 0]$$

$$y_1 = [0.217, 0.062, -0.093]$$

$$\text{softmax}(y_1):$$

$$\exp(y_1) = [1.242, 1.064, 0.911]$$

$$\Sigma = 3.217$$

$$P_1 = [1.242/3.217, 1.064/3.217, 0.911/3.217]$$

$$P_1 = [0.386, 0.331, 0.283]$$

$$P(\text{next} = 'a') = 0.386$$

$$P(\text{next} = 'b') = 0.331$$

$$P(\text{next} = 'c') = 0.283$$

c) Loss if target is 'b' (index 1):

```
L_1 = -log(P_1[1])  
L_1 = -log(0.331)  
L_1 = 1.106
```

d) Hidden state h_2 after sequence "ab":

Input 'b' $\rightarrow$ index 1

```
e_2 = W_embed[1] = [0.3, -0.1]  
  
h_2 = tanh(W_xh^T · e_2 + W_hh^T · h_1 + b_h)  
  
W_xh^T · e_2 =  $\begin{bmatrix} 0.6 & 0.1 \\ -0.2 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0.3 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.17 \\ -0.11 \end{bmatrix}$   
  
W_hh^T · h_1 =  $\begin{bmatrix} 0.4 & -0.2 \\ 0.3 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0.310 \\ 0.000 \end{bmatrix} = \begin{bmatrix} 0.124 \\ 0.093 \end{bmatrix}$   
  
h_2 = tanh([0.17, -0.11] + [0.124, 0.093] + [0.0, 0.0])  
h_2 = tanh([0.294, -0.017])  
h_2 = [0.286, -0.017]
```

Answer Summary:

- $h_1 = [0.310, 0.000]$
- $P(\text{next_char}) = [0.386, 0.331, 0.283]$
- Loss = 1.106
- $h_2 = [0.286, -0.017]$

2. Image Captioning

2.1 Problem Statement

Task: Generate a caption for an image **Architecture:** CNN + RNN

Process:

1. CNN extracts image features $\rightarrow$ single feature vector
2. RNN generates caption word-by-word using image features

2.2 Architecture Details

Image → CNN (e.g., ResNet) → Feature Vector (4096-d)
↓
RNN Decoder
↓
Word Sequence Output

Example:

Input: Image of a cat on a couch
Output: "a cat sitting on a couch"

2.3 Step-by-Step Example

Setup

Image Feature: $v = [4096\text{-dimensional vector from CNN}]$ For simplicity, use $v = [0.8, 0.3, -0.5, 0.2]$ (4-d)

Vocabulary: $['<START>', 'a', 'cat', 'on', 'couch', '<END>']$ (6 words)

Word Indices:

$<START> \rightarrow 0$
 $a \rightarrow 1$
 $cat \rightarrow 2$
 $on \rightarrow 3$
 $couch \rightarrow 4$
 $<END> \rightarrow 5$

Target Caption: "a cat on couch"

Dimensions:

- Image feature: 4
- Hidden state: 3
- Vocabulary size: 6
- Word embedding: 2

Step 1: Initialize RNN with Image Features

Project image features to hidden state:

$$W_{ih} (4 \times 3) = \begin{bmatrix} 0.5 & -0.2 & 0.3 \\ 0.2 & 0.4 & -0.1 \\ -0.3 & 0.1 & 0.5 \\ 0.1 & 0.3 & -0.2 \end{bmatrix}$$

$$b_h = [0.0, 0.0, 0.0]$$

$$h_0 = \tanh(W_{ih}^T \cdot v + b_h)$$

$$W_{ih}^T \cdot v = \begin{bmatrix} 0.5 & 0.2 & -0.3 & 0.1 \\ -0.2 & 0.4 & 0.1 & 0.3 \\ 0.3 & -0.1 & 0.5 & -0.2 \end{bmatrix} \times \begin{bmatrix} 0.8 \\ 0.3 \\ -0.5 \\ 0.2 \end{bmatrix}$$

Row 1: $0.5(0.8) + 0.2(0.3) + (-0.3)(-0.5) + 0.1(0.2) = 0.4 + 0.06 + 0.15 + 0.02 = 0.63$

Row 2: $-0.2(0.8) + 0.4(0.3) + 0.1(-0.5) + 0.3(0.2) = -0.16 + 0.12 - 0.05 + 0.06 = -0.03$

Row 3: $0.3(0.8) + (-0.1)(0.3) + 0.5(-0.5) + (-0.2)(0.2) = 0.24 - 0.03 - 0.25 - 0.04 = -0.08$

$$h_0 = \tanh([0.63, -0.03, -0.08])$$

$$h_0 = [0.558, -0.030, -0.080]$$

Interpretation: Image features encoded in initial hidden state

Step 2: Generate First Word

Input: <START> token (index 0)

Word Embedding:

$$W_{emb} (6 \times 2) = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & -0.1 \\ -0.2 & 0.4 \\ 0.5 & 0.1 \\ 0.2 & -0.3 \\ 0.0 & 0.0 \end{bmatrix} \begin{matrix} \# \text{ <START>} \\ \# \text{ a} \\ \# \text{ cat} \\ \# \text{ on} \\ \# \text{ couch} \\ \# \text{ <END>} \end{matrix}$$

$$e_1 = W_{emb}[0] = [0.1, 0.2]$$

RNN Computation:

$$W_{xh} (2 \times 3) = \begin{bmatrix} 0.6 & -0.3 & 0.2 \\ 0.2 & 0.5 & -0.1 \end{bmatrix}$$

$$W_{hh} (3 \times 3) = \begin{bmatrix} 0.4 & -0.2 & 0.1 \\ 0.3 & 0.5 & -0.2 \\ -0.1 & 0.2 & 0.4 \end{bmatrix}$$

$$h_1 = \tanh(W_{xh}^T \cdot e_1 + W_{hh}^T \cdot h_0 + b_h)$$

$$W_{xh}^T \cdot e_1 = \begin{bmatrix} 0.6 & 0.2 \\ -0.3 & 0.5 \\ 0.2 & -0.1 \end{bmatrix} \times \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.10 \\ 0.07 \\ 0.00 \end{bmatrix}$$

$$W_{hh}^T \cdot h_0 = \begin{bmatrix} 0.4 & 0.3 & -0.1 \\ -0.2 & 0.5 & 0.2 \\ 0.1 & -0.2 & 0.4 \end{bmatrix} \times \begin{bmatrix} 0.558 \\ -0.030 \\ -0.080 \end{bmatrix}$$

$$\text{Row 1: } 0.4(0.558) + 0.3(-0.030) + (-0.1)(-0.080) = 0.223 - 0.009 + 0.008 = 0.222$$

$$\text{Row 2: } -0.2(0.558) + 0.5(-0.030) + 0.2(-0.080) = -0.112 - 0.015 - 0.016 = -0.143$$

$$\text{Row 3: } 0.1(0.558) + (-0.2)(-0.030) + 0.4(-0.080) = 0.056 + 0.006 - 0.032 = 0.030$$

$$h_1 = \tanh([0.10, 0.07, 0.00] + [0.222, -0.143, 0.030] + [0, 0, 0])$$

$$h_1 = \tanh([0.322, -0.073, 0.030])$$

$$h_1 = [0.311, -0.073, 0.030]$$

Output Word Prediction:

$$W_{hy} (3 \times 6) = \begin{bmatrix} 0.5 & 0.2 & -0.3 & 0.4 & 0.1 & -0.2 \\ 0.3 & -0.1 & 0.5 & -0.2 & 0.4 & 0.2 \\ -0.2 & 0.4 & 0.1 & 0.3 & -0.1 & 0.5 \end{bmatrix}$$

$$y_1 = W_{hy}^{AT} \cdot h_1 + b_y$$

$$W_{hy}^{AT} \cdot h_1 = \begin{bmatrix} 0.5 & 0.3 & -0.2 \\ 0.2 & -0.1 & 0.4 \\ -0.3 & 0.5 & 0.1 \\ 0.4 & -0.2 & 0.3 \\ 0.1 & 0.4 & -0.1 \\ -0.2 & 0.2 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0.311 \\ -0.073 \\ 0.030 \end{bmatrix}$$

Row 1 (<START>): $0.5(0.311) + 0.3(-0.073) + (-0.2)(0.030) = 0.156 - 0.022 - 0.006 = 0.128$

Row 2 (a): $0.2(0.311) + (-0.1)(-0.073) + 0.4(0.030) = 0.062 + 0.007 + 0.012 = 0.081$

Row 3 (cat): $-0.3(0.311) + 0.5(-0.073) + 0.1(0.030) = -0.093 - 0.037 + 0.003 = -0.127$

Row 4 (on): $0.4(0.311) + (-0.2)(-0.073) + 0.3(0.030) = 0.124 + 0.015 + 0.009 = 0.148$

Row 5 (couch): $0.1(0.311) + 0.4(-0.073) + (-0.1)(0.030) = 0.031 - 0.029 - 0.003 = -0.001$

Row 6 (<END>): $-0.2(0.311) + 0.2(-0.073) + 0.5(0.030) = -0.062 - 0.015 + 0.015 = -0.062$

$$y_1 = [0.128, 0.081, -0.127, 0.148, -0.001, -0.062]$$

Softmax:

$$\exp(y_1) = [1.137, 1.084, 0.881, 1.159, 0.999, 0.940]$$

$$\Sigma = 6.200$$

$$P_1 = [0.183, 0.175, 0.142, 0.187, 0.161, 0.152]$$

Predicted word: "on" (index 3, highest probability 0.187) **Target word:** "a" (index 1)

Loss: $L_1 = -\log(0.175) = 1.743$

Step 3: Generate Second Word

Input: Previous output "a" (index 1) - could be:

- During training: ground truth "a"
- During inference: predicted word "on"

Using ground truth (Teacher Forcing):

```
e_2 = W_emb[1] = [0.3, -0.1]
```

```
h_2 = tanh(W_xh^T · e_2 + W_hh^T · h_1 + b_h)
```

```
W_xh^T · e_2 = [[0.6, 0.2],      [[0.3],      [[0.16],
                [-0.3, 0.5],    ×  [-0.1]] =  [-0.14],
                [0.2, -0.1]]      [0.07]]
```

```
W_hh^T · h_1 = [[0.4, 0.3, -0.1],    [[0.311],
                [-0.2, 0.5, 0.2],      ×  [-0.073],
                [0.1, -0.2, 0.4]]      [0.030]]
```

```
Row 1: 0.4(0.311) + 0.3(-0.073) + (-0.1)(0.030) = 0.124 - 0.022 - 0.003 = 0.099
```

```
Row 2: -0.2(0.311) + 0.5(-0.073) + 0.2(0.030) = -0.062 - 0.037 + 0.006 = -0.093
```

```
Row 3: 0.1(0.311) + (-0.2)(-0.073) + 0.4(0.030) = 0.031 + 0.015 + 0.012 = 0.058
```

```
h_2 = tanh([0.16, -0.14, 0.07] + [0.099, -0.093, 0.058] + [0, 0, 0])
```

```
h_2 = tanh([0.259, -0.233, 0.128])
```

```
h_2 = [0.253, -0.229, 0.127]
```

Output:

```
y_2 = W_hy^T · h_2
```

```
(Similar calculation)
```

```
P_2 = softmax(y_2)
```

```
Predict: "cat" (hopefully highest probability)
```

Step 4: Continue Until <END> Token

Continue this process:

```
t=1: <START> → predict "a"
```

```
t=2: a → predict "cat"
```

```
t=3: cat → predict "on"
```

```
t=4: on → predict "couch"
```

```
t=5: couch → predict <END>
```

2.4 Complete Example Question with Answer

Question 2: Image Captioning Forward Pass

Given:

- Image feature vector: $v = [0.6, 0.4, -0.2]$ (3-d)
- Vocabulary: ['<START>', 'dog', 'runs', '<END>'] (4 words)
- Target caption: "dog runs"
- Hidden dimension: 2
- Embedding dimension: 2

Weights:

```
W_ih = [[0.7, 0.3],
        [0.2, -0.4],
        [-0.1, 0.5]]

W_emb = [[0.2, 0.1],    # <START>
        [0.4, -0.2],   # dog
        [-0.3, 0.3],   # runs
        [0.0, 0.0]]    # <END>

W_xh = [[0.5, -0.2],
        [0.3, 0.4]]

W_hh = [[0.6, 0.2],
        [-0.3, 0.5]]

W_hy = [[0.8, 0.3, -0.4, 0.2],
        [0.1, -0.5, 0.6, 0.4]]

All biases = 0
```

Tasks: a) Calculate initial hidden state h_0 from image features b) Calculate h_1 after inputting <START> token c) Calculate output probabilities $P(\text{next_word})$ at $t=1$ d) What is the predicted first word?

Solution:

a) Initial hidden state h_0 :

$$h_0 = \tanh(W_{ih}^T \cdot v)$$

$$W_{ih}^T \cdot v = \begin{bmatrix} 0.7 & 0.2 & -0.1 \\ 0.3 & -0.4 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0.6 \\ 0.4 \\ -0.2 \end{bmatrix}$$

$$\text{Row 1: } 0.7(0.6) + 0.2(0.4) + (-0.1)(-0.2) = 0.42 + 0.08 + 0.02 = 0.52$$

$$\text{Row 2: } 0.3(0.6) + (-0.4)(0.4) + 0.5(-0.2) = 0.18 - 0.16 - 0.10 = -0.08$$

$$h_0 = \tanh([0.52, -0.08])$$

$$h_0 = [0.478, -0.080]$$

b) Hidden state h_1 after <START>:

$$e_1 = W_{emb}[0] = [0.2, 0.1]$$

$$h_1 = \tanh(W_{xh}^T \cdot e_1 + W_{hh}^T \cdot h_0)$$

$$W_{xh}^T \cdot e_1 = \begin{bmatrix} 0.5 & 0.3 \\ -0.2 & 0.4 \end{bmatrix} \times \begin{bmatrix} 0.2 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.13 \\ 0.00 \end{bmatrix}$$

$$W_{hh}^T \cdot h_0 = \begin{bmatrix} 0.6 & -0.3 \\ 0.2 & 0.5 \end{bmatrix} \times \begin{bmatrix} 0.478 \\ -0.080 \end{bmatrix} = \begin{bmatrix} 0.311 \\ 0.056 \end{bmatrix}$$

$$h_1 = \tanh([0.13, 0.00] + [0.311, 0.056])$$

$$h_1 = \tanh([0.441, 0.056])$$

$$h_1 = [0.414, 0.056]$$

c) Output probabilities:

$$y_1 = W_{hy}^T \cdot h_1$$

$$W_{hy}^T = \begin{bmatrix} 0.8 & 0.1 \\ 0.3 & -0.5 \\ -0.4 & 0.6 \\ 0.2 & 0.4 \end{bmatrix}$$

$$W_{hy}^T \cdot h_1 = \begin{bmatrix} 0.8 & 0.1 \\ 0.3 & -0.5 \\ -0.4 & 0.6 \\ 0.2 & 0.4 \end{bmatrix} \times \begin{bmatrix} 0.414 \\ 0.056 \end{bmatrix} = \begin{bmatrix} 0.337 \\ 0.096 \\ -0.132 \\ 0.105 \end{bmatrix}$$

$$y_1 = [0.337, 0.096, -0.132, 0.105]$$

Softmax:

$$\exp(y_1) = [1.401, 1.101, 0.876, 1.111]$$

$$\Sigma = 4.489$$

$$P_1 = [0.312, 0.245, 0.195, 0.248]$$

$$P(<START>) = 0.312 \quad P(\text{dog}) = 0.245 \quad P(\text{runs}) = 0.195 \quad P(<END>) = 0.248$$

d) Predicted first word: <START> (index 0, probability 0.312)

Note: This is incorrect! The model incorrectly predicts <START> again instead of "dog". This shows the model needs more training.

3. Vanilla RNN Gradient Flow

3.1 Problem Statement

Understanding: Why do vanilla RNNs suffer from vanishing/exploding gradients?

Key Concept: During backpropagation through time (BPTT), gradients are multiplied many times, leading to exponential decay (vanishing) or growth (exploding).

3.2 Mathematical Analysis

RNN Equations

$$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$$

$$y_t = W_{hy} \cdot h_t + b_y$$

Gradient Flow

To update W_{hh} , we need $\partial L / \partial W_{hh}$

Chain Rule for gradient from h_t to h_0 :

$$\partial h_t / \partial h_0 = \partial h_t / \partial h_{t-1} \cdot \partial h_{t-1} / \partial h_{t-2} \cdot \dots \cdot \partial h_1 / \partial h_0$$

Each term:

$$\partial h_t / \partial h_{t-1} = \text{diag}(\tanh'(a_t)) \cdot W_{hh}$$

where $a_t = W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h$

Full gradient:

$$\partial h_t / \partial h_0 = \prod_{i=1}^t \text{diag}(\tanh'(a_i)) \cdot W_{hh}$$

3.3 Numerical Example: Vanishing Gradient

Setup:

```
Sequence length T = 4
Hidden dimension: 2
W_hh = [[0.3, 0.2],
        [0.1, 0.4]]
```

Assume $\tanh'(a_t) \approx 0.5$ (typical value)

Step-by-Step Gradient Calculation

Time t=1:

$$\begin{aligned} \partial h_1 / \partial h_0 &= \text{diag}([0.5, 0.5]) \cdot W_{hh} \\ &= \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \cdot \begin{bmatrix} 0.3 & 0.2 \\ 0.1 & 0.4 \end{bmatrix} \\ &= \begin{bmatrix} 0.15 & 0.10 \\ 0.05 & 0.20 \end{bmatrix} \end{aligned}$$

Norm: $\|\partial h_1 / \partial h_0\| \approx 0.28$

Time t=2:

$$\partial h_2 / \partial h_0 = \partial h_2 / \partial h_1 \cdot \partial h_1 / \partial h_0$$

$$\partial h_2 / \partial h_1 = \begin{bmatrix} 0.15, & 0.10 \\ 0.05, & 0.20 \end{bmatrix}$$

$$\partial h_2 / \partial h_0 = \begin{bmatrix} 0.15, & 0.10 \\ 0.05, & 0.20 \end{bmatrix} \cdot \begin{bmatrix} 0.15, & 0.10 \\ 0.05, & 0.20 \end{bmatrix}$$

$$\text{Row 1, Col 1: } 0.15(0.15) + 0.10(0.05) = 0.0225 + 0.005 = 0.0275$$

$$\text{Row 1, Col 2: } 0.15(0.10) + 0.10(0.20) = 0.015 + 0.020 = 0.035$$

$$\text{Row 2, Col 1: } 0.05(0.15) + 0.20(0.05) = 0.0075 + 0.010 = 0.0175$$

$$\text{Row 2, Col 2: } 0.05(0.10) + 0.20(0.20) = 0.005 + 0.040 = 0.045$$

$$\partial h_2 / \partial h_0 = \begin{bmatrix} 0.0275, & 0.035 \\ 0.0175, & 0.045 \end{bmatrix}$$

Norm: $\|\partial h_2 / \partial h_0\| \approx 0.068$

Time t=3:

$$\partial h_3 / \partial h_0 = \partial h_3 / \partial h_2 \cdot \partial h_2 / \partial h_0$$

Following same calculation:

$$\partial h_3 / \partial h_0 \approx \begin{bmatrix} 0.0066, & 0.0095 \\ 0.0050, & 0.0115 \end{bmatrix}$$

Norm: $\|\partial h_3 / \partial h_0\| \approx 0.017$

Time t=4:

$$\partial h_4 / \partial h_0 \approx \begin{bmatrix} 0.0015, & 0.0023 \\ 0.0013, & 0.0028 \end{bmatrix}$$

Norm: $\|\partial h_4 / \partial h_0\| \approx 0.004$

Observation: Vanishing Gradient

t=0 to t=1: norm = 0.28

t=0 to t=2: norm = 0.068 (↓ 76%)

t=0 to t=3: norm = 0.017 (↓ 75%)

t=0 to t=4: norm = 0.004 (↓ 76%)

Each timestep, gradient shrinks by ~75%!

After 10 timesteps: $(0.25)^{10} \approx 0.00000095$

After 20 timesteps: $(0.25)^{20} \approx 0.00000000000009$

Result: Cannot learn long-term dependencies!

3.4 Numerical Example: Exploding Gradient

Setup:

$$W_{hh} = \begin{bmatrix} 2.0 & 0.5 \\ 0.3 & 1.8 \end{bmatrix}$$

Assume $\tanh'(a_t) \approx 1.0$ (near zero input, maximum derivative)

Step-by-Step Calculation

Time t=1:

$$\partial h_1 / \partial h_0 = I \cdot W_{hh} = \begin{bmatrix} 2.0 & 0.5 \\ 0.3 & 1.8 \end{bmatrix}$$

Norm: $\|\partial h_1 / \partial h_0\| \approx 2.7$

Time t=2:

$$\begin{aligned} \partial h_2 / \partial h_0 &= W_{hh} \cdot W_{hh} \\ &= \begin{bmatrix} 2.0 & 0.5 \\ 0.3 & 1.8 \end{bmatrix} \cdot \begin{bmatrix} 2.0 & 0.5 \\ 0.3 & 1.8 \end{bmatrix} \\ &= \begin{bmatrix} 4.15 & 1.90 \\ 1.14 & 3.39 \end{bmatrix} \end{aligned}$$

Norm: $\|\partial h_2 / \partial h_0\| \approx 5.8$

Time t=3:

$$\begin{aligned} \partial h_3 / \partial h_0 &= W_{hh} \cdot \partial h_2 / \partial h_0 \\ &= \begin{bmatrix} 10.87 & 5.50 \\ 3.36 & 6.60 \end{bmatrix} \end{aligned}$$

Norm: $\|\partial h_3 / \partial h_0\| \approx 13.1$

Time t=4:

$$\partial h_4 / \partial h_0 \approx \begin{bmatrix} [26.43, 14.88], \\ [8.47, 14.73] \end{bmatrix}$$

Norm: $\|\partial h_4 / \partial h_0\| \approx 32.9$

Observation: Exploding Gradient

t=0 to t=1: norm = 2.7
t=0 to t=2: norm = 5.8 (↑ 115%)
t=0 to t=3: norm = 13.1 (↑ 126%)
t=0 to t=4: norm = 32.9 (↑ 151%)

Each timestep, gradient grows exponentially!

After 10 timesteps: $(2.2)^{10} \approx 5000$

After 20 timesteps: $(2.2)^{20} \approx 26$ million

Result:

- Weights become huge
- Numerical overflow (NaN)
- Training unstable

3.5 Complete Example Question with Answer

Question 3: Gradient Flow Analysis

Given a vanilla RNN with:

- Hidden dimension: 3
- Sequence length: $T = 3$
- W_{hh} (recurrent weight matrix):

$$W_{hh} = \begin{bmatrix} [0.4, 0.2, 0.1], \\ [0.3, 0.5, 0.2], \\ [0.1, 0.2, 0.3] \end{bmatrix}$$

Assume $\tanh'(a_t) = \text{diag}([0.6, 0.5, 0.4])$ for all timesteps.

Tasks: a) Calculate $\partial h_1 / \partial h_0$ b) Calculate $\partial h_2 / \partial h_0$ c) Calculate $\|\partial h_2 / \partial h_0\| / \|\partial h_1 / \partial h_0\|$ (ratio of gradient norms) d) If this ratio continues, what happens to gradients over 50 timesteps?

Solution:

a) Calculate $\partial h_1 / \partial h_0$:

$$\partial h_1 / \partial h_0 = \text{diag}(\tanh'(a_1)) \cdot W_{hh}$$

$$= \begin{bmatrix} 0.6 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.4 \end{bmatrix} \cdot \begin{bmatrix} 0.4 & 0.2 & 0.1 \\ 0.3 & 0.5 & 0.2 \\ 0.1 & 0.2 & 0.3 \end{bmatrix}$$

$$\text{Row 1: } [0.6 \times 0.4, 0.6 \times 0.2, 0.6 \times 0.1] = [0.24, 0.12, 0.06]$$

$$\text{Row 2: } [0.5 \times 0.3, 0.5 \times 0.5, 0.5 \times 0.2] = [0.15, 0.25, 0.10]$$

$$\text{Row 3: } [0.4 \times 0.1, 0.4 \times 0.2, 0.4 \times 0.3] = [0.04, 0.08, 0.12]$$

$$\partial h_1 / \partial h_0 = \begin{bmatrix} 0.24 & 0.12 & 0.06 \\ 0.15 & 0.25 & 0.10 \\ 0.04 & 0.08 & 0.12 \end{bmatrix}$$

Norm (Frobenius norm):

$$\begin{aligned} ||\partial h_1 / \partial h_0|| &= \sqrt{\sum \text{all_elements}^2} \\ &= \sqrt{0.24^2 + 0.12^2 + 0.06^2 + 0.15^2 + 0.25^2 + 0.10^2 + 0.04^2 + 0.08^2 + 0.12^2} \\ &= \sqrt{0.0576 + 0.0144 + 0.0036 + 0.0225 + 0.0625 + 0.0100 + 0.0016 + 0.0064 + 0.0144} \\ &= \sqrt{0.193} \\ &= 0.439 \end{aligned}$$

b) Calculate $\partial h_2 / \partial h_0$:

$$\partial h_2 / \partial h_0 = \partial h_2 / \partial h_1 \cdot \partial h_1 / \partial h_0$$

$$\partial h_2 / \partial h_1 = \begin{bmatrix} 0.24, & 0.12, & 0.06 \\ 0.15, & 0.25, & 0.10 \\ 0.04, & 0.08, & 0.12 \end{bmatrix}$$

$$\partial h_2 / \partial h_0 = \begin{bmatrix} 0.24, & 0.12, & 0.06 \\ 0.15, & 0.25, & 0.10 \\ 0.04, & 0.08, & 0.12 \end{bmatrix} \cdot \begin{bmatrix} 0.24, & 0.12, & 0.06 \\ 0.15, & 0.25, & 0.10 \\ 0.04, & 0.08, & 0.12 \end{bmatrix}$$

$$\text{Row 1, Col 1: } 0.24(0.24) + 0.12(0.15) + 0.06(0.04) = 0.0576 + 0.018 + 0.0024 = 0.078$$

$$\text{Row 1, Col 2: } 0.24(0.12) + 0.12(0.25) + 0.06(0.08) = 0.0288 + 0.030 + 0.0048 = 0.064$$

$$\text{Row 1, Col 3: } 0.24(0.06) + 0.12(0.10) + 0.06(0.12) = 0.0144 + 0.012 + 0.0072 = 0.034$$

$$\text{Row 2, Col 1: } 0.15(0.24) + 0.25(0.15) + 0.10(0.04) = 0.036 + 0.0375 + 0.004 = 0.078$$

$$\text{Row 2, Col 2: } 0.15(0.12) + 0.25(0.25) + 0.10(0.08) = 0.018 + 0.0625 + 0.008 = 0.089$$

$$\text{Row 2, Col 3: } 0.15(0.06) + 0.25(0.10) + 0.10(0.12) = 0.009 + 0.025 + 0.012 = 0.046$$

$$\text{Row 3, Col 1: } 0.04(0.24) + 0.08(0.15) + 0.12(0.04) = 0.0096 + 0.012 + 0.0048 = 0.026$$

$$\text{Row 3, Col 2: } 0.04(0.12) + 0.08(0.25) + 0.12(0.08) = 0.0048 + 0.020 + 0.0096 = 0.034$$

$$\text{Row 3, Col 3: } 0.04(0.06) + 0.08(0.10) + 0.12(0.12) = 0.0024 + 0.008 + 0.0144 = 0.025$$

$$\partial h_2 / \partial h_0 = \begin{bmatrix} 0.078, & 0.064, & 0.034 \\ 0.078, & 0.089, & 0.046 \\ 0.026, & 0.034, & 0.025 \end{bmatrix}$$

Norm:

$$\begin{aligned} ||\partial h_2 / \partial h_0|| &= \sqrt{0.078^2 + 0.064^2 + 0.034^2 + 0.078^2 + 0.089^2 + 0.046^2 + 0.026^2 + 0.034^2 + 0.025^2} \\ &= \sqrt{0.00608 + 0.00410 + 0.00116 + 0.00608 + 0.00792 + 0.00212 + 0.00068 + 0.00116 + 0.00063} \\ &= \sqrt{0.02993} \\ &= 0.173 \end{aligned}$$

c) Gradient norm ratio:

$$\begin{aligned}\text{Ratio} &= ||\partial h_2 / \partial h_0|| / ||\partial h_1 / \partial h_0|| \\ &= 0.173 / 0.439 \\ &= 0.394 \\ &\approx 0.4\end{aligned}$$

Interpretation: Gradient shrinks to 40% each timestep.

d) After 50 timesteps:

If ratio = 0.4 per step:
After 50 steps: $(0.4)^{50}$

$$(0.4)^{10} \approx 0.0001$$
$$(0.4)^{50} = ((0.4)^{10})^5 \approx (0.0001)^5 \approx 10^{-20}$$

Result: Gradient becomes essentially zero! The network cannot learn dependencies beyond a few timesteps.

This is the vanishing gradient problem!

3.6 Solutions to Gradient Problems

1. Gradient Clipping (for exploding gradients)

```
python

def clip_gradient(grad, max_norm=5.0):
    norm = ||grad||
    if norm > max_norm:
        grad = grad * (max_norm / norm)
    return grad
```

Example:

$$\begin{aligned}\text{grad} &= [10, 15, 20] \\ ||\text{grad}|| &= \sqrt{10^2 + 15^2 + 20^2} = \sqrt{725} \approx 26.9\end{aligned}$$

Since $26.9 > 5.0$:

$$\begin{aligned}\text{grad_clipped} &= [10, 15, 20] * (5.0 / 26.9) \\ &= [1.86, 2.79, 3.72]\end{aligned}$$

2. LSTM/GRU Architecture (for vanishing gradients)

Key Idea: Additive updates instead of multiplicative

LSTM Cell State Update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Gradient Flow:

$$\partial C_t / \partial C_{t-1} = f_t$$

No matrix multiplication! Just element-wise multiplication by forget gate.

If $f_t \approx 1$, gradient flows unchanged through time!

Summary

Character-Level Language Model

- **Input:** Previous characters
- **Output:** Probability distribution over next character
- **Key:** One-hot encoding $\rightarrow$ Embedding $\rightarrow$ RNN $\rightarrow$ Softmax
- **Sampling:** Generate text character by character

Image Captioning

- **Input:** Image features from CNN
- **Output:** Word sequence
- **Key:** CNN encodes image $\rightarrow$ RNN decodes to caption
- **Training:** Teacher forcing (use ground truth)
- **Testing:** Autoregressive (use predictions)

Vanilla RNN Gradient Flow

- **Problem:** Gradients vanish or explode through time
- **Cause:** Repeated matrix multiplication
- **Vanishing:** Weights $< 1 \rightarrow$ gradient $\rightarrow 0$
- **Exploding:** Weights $> 1 \rightarrow$ gradient $\rightarrow \infty$

- **Solutions:**
 - Gradient clipping (exploding)
 - LSTM/GRU (vanishing)
-

Practice Problems

Problem 1: Character-Level Model

Train a character RNN on "cat" with vocabulary ['c', 'a', 't']. Calculate hidden states for all timesteps with given weights.

Problem 2: Image Captioning Loss

Given image features and caption "dog runs", calculate total loss across all timesteps.

Problem 3: Gradient Flow

Prove mathematically that if max eigenvalue of W_{hh} is λ , then $\|\partial h_t / \partial h_0\| \approx \lambda^t$.

Problem 4: Comparison

Compare parameter count and gradient flow between vanilla RNN and LSTM for same hidden dimension.

Good luck with your exam! 🎓